



University of sigmas

Oiowe chady

Adam Biel, Mateusz Jurach, Karol Źeleźnik

2024-09-18

- 1
- 2
- 3
- 4
- Templatki
- Matma
- Struktury danych
- Optymalizacje

Templatki (1)

.bashrc	11 lines
<pre>c() { g++ -std=c++20 -Wall -Wno-sign-conversion -Wextra -Wshadow - Wconversion -Wfloat-equal \ -fsanitize=address,undefined -DLOCAL -DDEBUG -D_GLIBCXX_DEBUG -ggdb3 \$1.cpp -o \$1 } nc() { g++ -std=c++20 -DLOCAL -O3 -static \$1.cpp -o \$1 } alias cp='cp -i' alias mv='mv -i'</pre>	
.vimrc	7 lines
<pre>set nocompatible filetype indent on set autoindent set number syntax on set ts=4 sw=4 set rnu</pre>	
main.cpp	24 lines
<pre>#include <bits/stdc++.h> using namespace std; #ifdef DEBUG auto&operator<<(auto &o, pair<auto, auto> p) {o << "(" << p. first << ", " << p.second << ")"; return o;} auto&operator<<(auto &o, auto x) {o<<"{"; for(auto e : x) o<<e <<" "; return o<<"}";} #define debug(X) cerr << "["#X"]": " << X << '\n'; #else #define cerr if(0)cout #define debug(X) ; #endif using ll = long long; #define all(v) (v).begin(), (v).end() #define ssize(x) int(x.size()) #define fi first #define se second #define mp make_pair #define eb emplace_back int main() { ios_base::sync_with_stdio(false); cin.tie(nullptr); return 0; }</pre>	

gen.cpp	1
Opis: Dodatek do generowania testów, lub innego losowania. Ziarno można zmieniać np. <code>rng.seed(atoi(argv[1]))</code> ;	
1	
<pre>mt19937 rng(random_device{}()); int rd(int l, int r) { return uniform_int_distribution<int>(l, r)(rng); }</pre>	
2	
4	
Matma (2)	
ext-euclid.cpp	
Opis: $O(\log n)$. Zwraca trójkę $[d, x, y]$, że $d = \gcd(a, b)$ i $ax + by = d$.	
<pre>tuple<ll, ll, ll> euclid(ll a, ll b) { if (b == 0) return make_tuple(a, 1, 0); auto [d, x, y] = euclid(b, a%b); return make_tuple(d, y, x - (a/b) * y); }</pre>	
mod.cpp	
Opis: operacje mod 998244353.	
<pre>const int mod = 998244353; int add(int a, int b) { a += b; return a >= mod ? a-mod : a; } int sub(int a, int b) { return add(a, mod-b); } int mul(int a, int b) { return int(((ll)a*b)%mod); } int fpow(int a, int b) { int res = 1; while (b) { if (b&1) { res = mul(a, res); b--; } a = mul(a, a); b /= 2; } return res; } int inv(int x) { return fpow(x, mod-2); }</pre>	
ntt.cpp	
Opis: Mnożenie wielomianów mod 998244353. Złożoność $O(\log n)$.	
<pre>#include "mod.cpp" //keep-include const int g = 3; const int g_1 = 332748118; using vi = vector<int>; void ntt(vi &a, int n, bool invert = false) { assert((n&(n-1)) == 0); a.resize(n); for (int i = 1, j = 0; i < n; ++i) {</pre>	

<pre>int bit = n/2; for (; bit&j; bit /= 2) j ^= bit; j ^= bit; if (i < j) swap(a[i], a[j]); }</pre>	
<pre>for (int len = 2; len <= n; len *= 2) { int r_len = fpow(invert ? g_1 : g, (mod-1) / len); for (int i = 0; i < n; i += len) { int r = 1; for (int j = 0; j < len/2; ++j, r = mul(r, r_len)) { int u = a[i + j], v = mul(r, a[i + j + len/2]); a[i+j] = add(u, v); a[i+j+len/2] = sub(u, v); } } }</pre>	
<pre>if (invert) { int inv_n = inv(n); for (int &x : a) x = mul(x, inv_n); } }</pre>	
<pre>vi conv(vi a, vi b) { if (a.empty() b.empty()) return {}; int n = ssize(a) + ssize(b) - 1; int sz = 1; while (sz < ssize(a) + ssize(b) - 1) sz *= 2; ntt(a, sz); ntt(b, sz); for (int i = 0; i < sz; ++i) a[i] = mul(a[i], b[i]); ntt(a, sz, true); a.resize(n); return a; }</pre>	
poly.cpp	
Opis: Operacje na wielmianach mod 998244353. Można przepisać co się chce, przy funkcjach jest napisane co potrzeba. deriv, integr w $O(n)$. inv, log, exp w $O(n \log n)$.	
<pre>#include "ntt.cpp" //keep-include using vi = vector<int>; vi mod_xn(const vi &v, int n) { //ZAWSZE return vi(v.begin(), v.begin() + min(n, ssize(v))); } vi add(vi a, const vi &b) { a.resize(max(ssize(a), ssize(b))); for (int i = 0; i < ssize(b); ++i) a[i] = add(a[i], b[i]); return a; } vi sub(vi a, const vi &b) { //ZAWSZE a.resize(max(ssize(a), ssize(b))); for (int i = 0; i < ssize(b); ++i) a[i] = sub(a[i], b[i]); return a; } vi deriv(vi p) { for (int i = 1; i < ssize(p); ++i) p[i] = mul(p[i], i); if (ssize(p)) p.erase(p.begin()); return p; }</pre>	

```

}

vi integr(vi p) {
    for (int i = 0; i < ssize(p); ++i)
        p[i] = mul(p[i], inv(i+1));
    p.insert(p.begin(), 0);
    return p;
}

vi inv(vi p, int n) {
    if (n == 1)
        return {inv(p[0])};
    p.resize(n);
    vi _p = p;
    for (int i = 1; i < n; i += 2) _p[i] = sub(0, p[i]);

    vi t = mod_xn(conv(p, _p), n);
    for (int i = 1; i < n; i += 2) assert(t[i] == 0);
    for (int i = 0; i < n; i += 2)
        t[i/2] = t[i];
    t = inv(t, (n+1)/2);
    t.resize(n);
    for (int i = (n-1)/2; i >= 0; --i) {
        t[2*i] = t[i];
        if(i) t[i] = 0;
    }
    return mod_xn(conv(_p, t), n);
}

vi log(const vi &p, int n) { //DERIV, INV, INTEGR
    assert(ssize(p) && p[0] == 1);
    return integr(mod_xn(conv(deriv(mod_xn(p, n)), inv(p, n)), n
        -1));
}

vi exp(const vi &p, int n) { //ADD, LOG
    assert(p.empty() || p[0] == 0);
    vi q = {1};
    for (int x = 1; x < n; x *= 2)
        q = mod_xn(conv(q, add(sub({1}, log(q, 2*x)), mod_xn(p, 2*x
            )), 2*x);
    q.resize(n);
    return q;
}
```

poly2d.cpp
Opis: Operacje na wielmianach dwóch zmiennych mod 998244353. Mnożenie w $O(mn \log(m+n))$. Dzielenie zwraca vector współczynników przy x^{n-1} , w czasie $O(mn \log^2(n+m))$.

```
#include "poly.cpp" //keep-include

void ntt(vector<vi> &a, int n, int m, bool inverse = false) {
    a.resize(n);
    for (int i = 0; i < ssize(a); ++i)
        ntt(a[i], m, inverse);
    for (int j = 0; j < ssize(a[0]); ++j) {
        vi c;
        for (int i = 0; i < ssize(a); ++i)
            c.eb(a[i][j]);
        ntt(c, n, inverse);
        for (int i = 0; i < ssize(a); ++i)
            a[i][j] = c[i];
    }
}

vector<vi> conv(vector<vi> a, vector<vi> b, int n, int m) {
    if (a.empty() || b.empty()) return {};
    int l1 = ssize(a) + ssize(b) - 1, sz1 = 1 << __lg(2*l1-1);
```

```

    int l2 = ssize(a[0]) + ssize(b[0]) - 1, sz2 = 1 << __lg(2*l2
        -1);

    ntt(a, sz1, sz2);
    ntt(b, sz1, sz2);
    for (int i = 0; i < sz1; ++i)
        for (int j = 0; j < sz2; ++j) a[i][j] = mul(a[i][j], b[i][j
            ]);
    ntt(a, sz1, sz2, true);
    a.resize(n);
    for (int i = 0; i < n; ++i) a[i].resize(m);
    return a;
}

vi div(vector<vi> b, vector<vi> a, int n, int m) {
    if (n == 1) {
        return conv(b[0], inv(a[0], m));
    }
    vector<vi> q = a;
    for (int i = 1; i < n; i += 2)
        for (int j = 0; j < ssize(a[i]); ++j) q[i][j] = sub(0, q[i
            ][j]);

    vector<vi> v = conv(a, q, n, 2*m);
    debug(v);
    q = conv(b, q, n, 2*m);
    for (int i = 0; i < ssize(v); i += 2) {
        for (int j = 0; j < ssize(v[i]); ++j) {
            v[i/2][j] = v[i][j];
        }
    }
    v.resize((n+1)/2);

    if (n%2 == 0) {
        for (int i = 1; i < n; i += 2)
            for (int j = 0; j < 2*m; ++j)
                q[i/2][j] = q[i][j];
        q.resize(n/2);
        return div(q, v, n/2, 2*m);
    }
    else {
        for (int i = 0; i < n; i += 2)
            for (int j = 0; j < 2*m; ++j)
                q[i/2][j] = q[i][j];
        q.resize((n+1)/2);
        return div(q, v, (n+1)/2, 2*m);
    }
}

}

Node() {
    low = 0, high = ALPH;
```

Struktury danych (3)

```

wavelet.cpp
Opis:  $O(\log^2 \text{ALPH})$  czasowo,  $O(n \log \text{ALPH})$  pamięciowo. Constructor
przyjmuje ciąg na którym rozpinamy drzewo, rank(l, r, k) zwraca k-tą
najmniejszą liczbę na przedziale [l;r]. (Uwaga!) pamięć około 262 Mb, dla
 $N, Q \leq 2 \cdot 10^5$ .

//funkcja cnt jest w  $O(\log)$  co daje cale operacje w  $O(\log^2)$ ,
da sie cnt w czasie statym — do dopisania
struct wavelet_tree {
    const static int ALPH = (1<<18)-1;
    struct Node {
        int low, high, mid;
        vector<int> v;
        Node *left, *right;

        Node() {
            low = 0, high = ALPH;
```

```

        left = right = nullptr;
        mid = (low+high)/2;
    }

    Node (const vector<int> &x, const vector<int> &t, int a =
        0, int b = ALPH): low(a), high(b), v(x) {
        mid = ((l1)low+high)/2;
        if(ssize(x) && low != high) {
            vector<int> div[2];
            for(auto e : v) div[t[e]<=mid].eb(e);
            if(ssize(div[1])) left = new Node(div[1], t, low, mid);
            else left = nullptr;
            if(ssize(div[0])) right = new Node(div[0], t, mid+1,
                high);
            else right = nullptr;
        }
        else {
            left = right = nullptr;
        }
    }
};

Node *root;
wavelet_tree(const vector<int> &b) {
    vector<int> idx;
    REP(i, ssize(b)) idx.eb(i);
    root = new Node(idx, b);
};

int cnt(Node *v, int l, int r) {
    if(v == nullptr) return 0;
    return int(upper_bound(all(v->v), r)-lower_bound(all(v->v),
        l));
}

int rank(int l, int r, int k, Node *v) {
    if(v->low == v->high) return v->low;
    if(cnt(v->left, l, r) >= k) return rank(l, r, k, v->left);
    else return rank(l, r, k-cnt(v->left, l, r), v->right);
}

int rank(int l, int r, int k) {
    return rank(l, r, k, root);
}

void clear(Node *v) {
    if(v->left != nullptr) clear(v->left);
    if(v->right != nullptr) clear(v->right);
    delete v;
}

~wavelet_tree() {
    clear(root);
}

};
```

treap.cpp
Opis: $O(\log n)$ czasowo. Treap z operacjami na (multi)secie. kth(t, k) zwraca k-tą najmniejszą liczbę.

```

namespace Treap {
    mt19937 rng(420);
    struct Node {
        Node *left, *right;
        int val, rank;
        int size;
        Node(): left(nullptr), right(nullptr), val(0), rank(int(rng
            ())), size(0) {}
    };
    using pNode = Node*;
```

```

Node pool[1'000'003];
int curr = 0;

int sz(pNode t) {
    return t == nullptr ? 0 : t->size;
}

void upd(pNode t) {
    t->size = sz(t->left) + sz(t->right) + 1;
}

pNode merge(pNode a, pNode b) {
    if(a == nullptr) return b;
    if(b == nullptr) return a;

    if(a->rank < b->rank) {
        a->right = merge(a->right, b);
        upd(a);
        return a;
    }
    else {
        b->left = merge(a, b->left);
        upd(b);
        return b;
    }
}

pair<pNode, pNode> split(pNode t, int x) {
    if(t == nullptr) return {nullptr, nullptr};

    if(t->val <= x) {
        auto [a, b] = split(t->right, x);
        t->right = a;
        upd(t);
        return {t, b};
    }
    else {
        auto [a, b] = split(t->left, x);
        t->left = b;
        upd(t);
        return {a, t};
    }
}

void insert(pNode &root, int x) {
    pNode u = pool + (curr++);
    u->val = x;
    u->size = 1;
    auto [a, b] = split(root, x);
    root = merge(merge(a, u), b);
}

bool find(pNode t, int x) {
    if(t == nullptr) return false;
    if(x == t->val) return true;
    if(x < t->val) return find(t->left, x);
    else return find(t->right, x);
}

void erase(pNode root, int x) {
    auto [ab, c] = split(root, x);
    auto [a, b] = split(ab, x-1);
    root = merge(a, c);
}

int kth(pNode t, int k) {
    assert(k >= 0);
    if(sz(t->left) == k-1) return t->val;
    if(k > sz(t->left)) return kth(t->right, k-1-sz(t->left));
}

```

```

        else return kth(t->left, k);
    }
}

```

implicit-treap.cpp

Opis: $O(\log n)$ czasowo. Implicit treap z podstawowymi operacjami, przykładowo odwracanie na przedziale (może być dowolna operacja).
 insert(root, k, x) wstawia element o wartości x w k -te miejsce tablicy.
 rev(root, l, r) odwraca przedział $[l; r]$ (indeksy od 0).

```

namespace Treap {
    mt19937 rng(420);
    struct Node {
        Node *left, *right;
        int rank;
        int size;
        bool rev;
        int val;
        Node(): left(nullptr), right(nullptr), rank(int(rng())),
            size(0), rev(false), val(0) {}
    };
    using pNode = Node*;

    Node pool[300'003];
    int curr = 0;

    int sz(pNode t) {
        return t == nullptr ? 0 : t->size;
    }

    void upd(pNode t) {
        t->size = sz(t->left) + sz(t->right) + 1;
    }

    void push_lazy(pNode t) {
        if(t->rev == false) return;
        if(t->left) t->left->rev = !t->left->rev;
        if(t->right) t->right->rev = !t->right->rev;
        swap(t->left, t->right);
        t->rev = false;
    }

    pNode merge(pNode a, pNode b) {
        if(a == nullptr) return b;
        if(b == nullptr) return a;

        push_lazy(a);
        push_lazy(b);
        if(a->rank < b->rank) {
            a->right = merge(a->right, b);
            upd(a);
            return a;
        }
        else {
            b->left = merge(a, b->left);
            upd(b);
            return b;
        }
    }

    pair<pNode, pNode> split(pNode t, int x) {
        if(t == nullptr) return {nullptr, nullptr};

        push_lazy(t);
        if(sz(t->left)+1 <= x) {
            auto [a, b] = split(t->right, x-(sz(t->left)+1));
            t->right = a;
            upd(t);
            return {t, b};
        }
    }
}

```

```

    }
    else {
        auto [a, b] = split(t->left, x);
        t->left = b;
        upd(t);
        return {a, t};
    }
}

pNode build(vector<int> a) {
    pNode root = nullptr;
    int n = ssize(a);
    for(int i = 0; i < n; ++i) {
        pNode u = pool + i;
        u->size = 1;
        u->val = a[i];
        root = merge(root, u);
    }
    curr += n;
    return root;
}

void insert(pNode &root, int k, int x) {
    auto [a, b] = split(root, k-1);
    Node *u = pool + (curr++);
    u->size = 1;
    u->val = x;
    root = merge(merge(a, u), b);
}

void reverse(pNode &root, int l, int r) {
    auto [ab, c] = split(root, r+1);
    auto [a, b] = split(ab, l);
    b->rev = !b->rev;
    root = merge(a, merge(b, c));
}

void print(pNode t) {
    if(t == nullptr) return;
    push_lazy(t);
    print(t->left);
    cout << t->val << ' ';
    print(t->right);
}
}

```

uf-rollbacks.cpp

Opis: $O(\log n)$ czasowo. Union find z roll backami – roll_back(x) cofa ostatnie x operacji unionn (union wierzchołków z tego samego zbioru się nie liczy).

```

struct dynamicUF {
    vector<int> e;
    dynamicUF(int n):e(n, -1) {}
    stack<array<int, 3>> s;

    int get(int a) {
        while(e[a] >= 0) a = e[a];
        return a;
    }

    int unionn(int a, int b) {
        a = get(a); b = get(b);
        if(a == b) return 0;
        if(-e[a] > -e[b]) swap(a, b);
        s.push({a, b, e[a]});
        e[b] += e[a];
        e[a] = b;
        return 1;
    }
}

```

```
    }

    void roll_back(int m = 1) {
        while (m--> 0) {
            auto [a, b, sz] = s.top();
            s.pop();
            e[b] -= sz;
            e[a] = sz;
        }
    }
};
```

fenwick-tree.cpp

Opis: (Binary Indexed Tree) $O(\log N)$, indeksowane od 0. Update w punkcie, zapytanie na przedziale

```
struct BIT {
    vector<int> tree;
    int N;
    BIT() {N = 0};
    BIT(int n) {
        N = n;
        tree.resize(N);
    }
    void resize(int n) {
        N = n;
        tree.resize(N);
    }
    void update(int p, int x) {
        for (++p; p <= N; p += (p&-p)) tree[p-1] += x;
    }
    int sum(int r) {
        int res = 0;
        for (; r > 0; r -= (r&-r)) res += tree[r-1];
        return res;
    }
    int query(int l, int r) {return sum(r+1)-sum(l);}
    int lower_bound(int sum) {
        int pos = 0;
        for (int pw = 1<<25; pw; pw<<=1) {
            int npos = pos+pw;
            if (npos <= N && tree[npos-1] < sum) {
                pos = npos; sum -= tree[pos-1];
            }
        }
        return pos;
    }
};
```

Optymalizacje (4)

fastio.cpp

Opis: fastio.

```
bool is_digit(char c) {
    return c >= '0';
}

int fastin(){
    int x = 0;
    char c;
    while (is_digit(c = getchar_unlocked())) {
        x = x * 10 + c - '0';
    }
    return x;
}

void write_int(unsigned long x) {
```

```
    char t[19];
    int i = 0;
    do {
        int d = x%10;
        t[i++] = '0'+d;
        x /= 10;
    } while (x > 0) ;

    while (--i >= 0) {
        putchar_unlocked(t[i]);
    }
    putchar_unlocked('\n');
}
```